

REPORT PROGETTO PSD

“COMUNE”

Scelta dell'ADT

Per la realizzazione di questo progetto è stato utilizzato l'ADT (Abstract Data Type) Lista “concatenata semplice”.

L'utilizzo di questo ADT risulta particolarmente adatto alla gestione delle segnalazioni poiché il numero di segnalazioni da registrare non è noto a priori e può variare durante l'esecuzione del programma.

L'utilizzo della Lista ci consente di avere una serie di vantaggi:

- Inserimento dinamico delle segnalazioni
- Gestione della memoria altamente ottimizzato ed efficace
- Inserimento rapido e ordinato delle nuove segnalazioni utilizzando l'inserimento in testa
- Eliminazione ottimizzata dei nodi senza lo spostamento di array

La Lista concatenata permette inoltre di effettuare rapidamente operazioni di ricerca e filtraggio grazie allo scorrimento sequenziale.

Risulta particolarmente efficace anche per la gestione delle priorità per le quali è stato usato una variabile numerica che varia da 1 a 5 in base all'urgenza, risultando molto comodo e rapido data la necessità di analizzare un semplice numero intero invece di una stringa.

La scelta di una lista concatenata semplice è stata preferita rispetto a strutture come array statici o liste doppiamente concatenate, poiché garantisce una gestione più semplice della struttura dati, un utilizzo dinamico della memoria e una maggiore efficienza nelle operazioni richieste dal progetto.

Progettazione

Il progetto è stato assemblato utilizzando una struttura modulare suddividendo il progetto in **4 file +1 file di test**, in modo da rendere il codice più ordinato, leggibile e manutenibile.

L'intero progetto pone base sulle **segnalazioni, conservate all'interno di una lista concatenata semplice**, nella quale ogni nodo rappresenta una singola segnalazione comunale.

Componenti principali:

- **main.c**

Il file "main.c" ha la funzione di **accesso principale** del programma.

Al suo interno vengono gestiti:

- Il menu interattivo
- L'interazione diretta con l'utente
- La scelta dell'operazione da eseguire
- La chiamata delle funzioni presenti negli altri file

Il file inoltre mantiene il puntatore alla testa della lista concatenata e utilizza un contatore per **generare automaticamente i codici identificativi** univoci delle segnalazioni.

- **segnalazioni.h**

Il file header "segnalazioni.h" contiene:

- La definizione della struttura dati Segnalazione
- i prototipi delle funzioni
- le librerie utilizzate per il funzionamento del progetto

Questo file ha la funzione di **collegare** tra di loro i diversi moduli del programma.

- segnalazioni.c

Il file “segnalazioni.c” contiene l'implementazione di tutte le funzioni operative del programma, le quali sono:

- L'inserimento delle segnalazioni
- Ricerca per codice
- Ricerca per categoria
- Aggiornamento dello stato
- Visualizzazione filtrata
- Gestione delle priorità
- Eliminazione di segnalazioni
- Generazione report
- Liberazione della memoria

Tutte le funzioni operano direttamente sulla lista grazie all'utilizzo dei puntatori “head” e “temp”.

- test.c

il file “test.c” esegue una serie di possibili casi preparati per verificare il corretto funzionamento di tutte le funzionalità del programma.

Il test permette di controllare:

- L'inserimento corretto dei nodi
- Funzionamento delle ricerche
- Aggiornamento degli stati
- Eliminazione delle segnalazioni
- Generazione dei report
- Corretta gestione della memoria

Alla fine dell'esecuzione dei test viene liberata la memoria e vengono aperti due getchar vuoti al fine di mantenere aperto il terminale e analizzare le operazioni eseguite.

- MAKEFILE

Il progetto include inoltre un file “Makefile” utilizzato per velocizzare e automatizzare il processo di compilazione del programma.

Il Makefile esegue le seguenti istruzioni:

- Compila automaticamente tutti i file principali
- genera gli eseguibili del “main” file e del “test” file
- Risparmia la compilazione manuale di ogni file necessario
- rimuove il file oggetto e gli eseguibili

Il Makefile funziona principalmente con l'utilizzo delle seguenti istruzioni:

- **make**: compila il progetto
- **make clean**: rimuove i file exe e oggetto
- **./main**: esegue il programma principale
- **./test**: esegue il programma di test

Interazione tra i vari componenti

Il programma funziona attraverso la collaborazione tra i diversi moduli:

Il file “main.c” ottiene gli input dall'utente, i quali vengono elaborati dalle funzioni richiamate nel file “segnalazioni.c”, utilizzando le librerie e le definizioni raccolte nel file “segnalazioni.h”, creando linearità nella struttura del programma.

Infine il file “test.c” esegue una verifica preimpostata per verificare il corretto funzionamento.

Specifica sintattica e semantica

Specifica dei tipi di dato

Specifica Sintattica:

- Tipo di riferimento: Segnalazione
- Tipi usati: Intero, Stringa

Specifica Semantica:

- Il tipo Segnalazione è una lista concatenata dove ogni elemento è composto da:
 - codice: intero
 - nome: stringa
 - categoria: stringa
 - descrizione: stringa
 - data: stringa
 - urgenza: intero
 - stato: intero

Specifica degli operatori

inserisciSegnalazione:

- **Specifica sintattica:** `(Segnalazione*, int*) -> Segnalazione*`
- **Specifica semantica:** `(head, contatoreId) = head (nuova)`
 - Pre: `contatoreId > 0`
 - Post: `head (nuova)` è la lista ottenuta inserendo in testa alla lista un nuovo elemento con `codice = contatoreId` ed i restanti campi inseriti dall'utente

visualizzaTutte:

- **Specifica sintattica:** `(Segnalazione*) -> void`
- **Specifica semantica:** `(head)`
 - Pre: `true`
 - post: Stampa a video in modo ordinato tutti gli elementi contenuti nella lista `head`

ricercaPerCategoria:

- **Specifica sintattica:** `(Segnalazione*, char*) -> void`
- **Specifica semantica:** `(head, categoriaCercata)`
 - Pre: `true`
 - post: Stampa a video tutti gli elementi presenti nella lista `head` a patto che la categoria appartenente a quella segnalazione coincida con quella specificata dall'utente

ricercaPerCodice:

- **Specifica sintattica:** (`Segnalazione*`, `int`)-> void
- **Specifica semantica:** (`head`, `codice`)
 - Pre: true
 - Post: Stampa a video tutti gli elementi della lista `head` a patto che il codice identificativo della segnalazione coincida con quello cercato dall'utente

aggiornaStato

- **Specifica sintattica:** (`Segnalazione*`, `int`, `int`)-> void
- **Specifica semantica:** (`head`, `codiceCercato`, `nuovoStato`)
 - Pre: `nuovoStato` >= 0 AND `nuovoStato` <= 2
 - post: Se esiste un elemento nella lista `head` con `codice` = `codiceCercato`, il valore `Stato` assume il valore di `nuovoStato`

visualizzaPerStato

- **Specifica sintattica:** (`Segnalazione*`, `int`)-> void
- **Specifica semantica:** (`head`, `stato`)
 - Pre: `stato` >= 0 AND `stato` <=2
 - Post: Stampa a video i soli elementi della lista `head` il cui campo `stato` coincida con lo stato cercato dall'utente

visualizzaUrgenti

- **Specifica sintattica:** (`Segnalazione*`)-> void
- **Specifica semantica:** (`head`)
 - Pre: true
 - post: Stampa a video tutti gli elementi della lista `head` il cui campo `urgenza` è >= 4

eliminaSegnalazione

- **Specifica sintattica:** (`Segnalazione*`, `int`)-> `Segnalazione*`
- **Specifica semantica:** (`head`, `codiceDaEliminare`)= `head`(`nuova`)
 - Pre: true
 - Post: Se esiste un elemento nella lista `head` con `codice` = `codiceDaEliminare`, la memoria della struttura puntata a `head`(`nuova`) viene deallocata e rappresenterà la lista aggiornata

generaReport

- **Specifica sintattica:** (`Segnalazione*`)-> void
- **Specifica semantica:** (`head`)
 - Pre: true
 - Post: Stampa a video il conteggio totale degli elementi nella lista `head` suddivisi per stato (Aperte, In lavorazione o Chiuse) e mostra i dettagli completi degli elementi il cui `Stato` = 0 OR `Stato` = 2

liberaMemoria

- **Specifica sintattica:** (*Segnalazione**) -> void
- **Specifica semantica:** (*head*)
 - Pre: true
 - Post: Dealloca la memoria dinamica occupata nella lista *head* in maniera iterativa

Razionale dei casi di test

Panoramica

Il file "test.c" è stato preparato per **testare la corretta esecuzione** delle funzioni applicate sull'ADT "Segnalazione" in maniera separata da ciò che è il programma principale onde evitare conflitti e procurare una maggiore chiarezza.

I casi di test sono stati progettati in modo tale da coprire ogni interazione possibile tra l'utente e il programma in maniera totalmente automatizzata strutturata nel seguente modo:

1) Setup dell'ambiente

Sono stati creati manualmente 3 segnalazioni contenenti dati sufficienti per coprire ogni possibile situazione:

- Si hanno tutti gli stati possibili (0, 1 e 2)
- Tre livelli di urgenza differenti (in modo tale da coprire le segnalazioni di alto livello di urgenza e basso livello di urgenza)
- Dati e categorie diversificate per un maggiore margine di test

2) Verifica operazioni di ricerca

Sono state utilizzate le funzioni di ricerca per ogni caso:

- Ricerca per codice esistente
- Ricerca di codice NON esistente
- Ricerca per stringa (Categoria)

3) Verifica operazioni di filtraggio

Viene testata la visualizzazione per Stato e per Urgenza con lo scopo di verificare il funzionamento degli "if" all'interno dei cicli implementati, ignorando i nodi che non soddisfano i parametri di ricerca

4) Verifica mutazione dati

Viene testata la modifica dello stato delle segnalazioni la quale viene verificata a schermo subito dopo la modifica tramite la funzione “visualizzaTutte”

5) Verifica dell'eliminazione e pulizia della memoria

- Eliminazione di un nodo viene testata creando un riferimento al nodo precedente permettendo in successione il collegamento dall'head precedente a quello successivo riorganizzando la lista
- Viene testato l'aggiornamento della lista tramite la stampa a video tramite un successivo "stampaReport"
- Viene liberata la memoria e verificata la corretta pulizia al fine di evitare memory leak.

Sistema Operativo utilizzato

L'intero progetto è stato realizzato su una macchina con sistema operativo Windows 11 con architettura basata su 64bit (x64)